

Architettura degli Elaboratori

L'architettura della CPU – Gestione dei control hazard



SAPIENZA
UNIVERSITÀ DI ROMA

Davide Polese

davide.polese@uniroma1.it

Special thanks and credits:

Alessandro Checco, Andrea Sterbini, Iacopo

Masi, Claudio di Ciccio

[S&PdC]

4.8



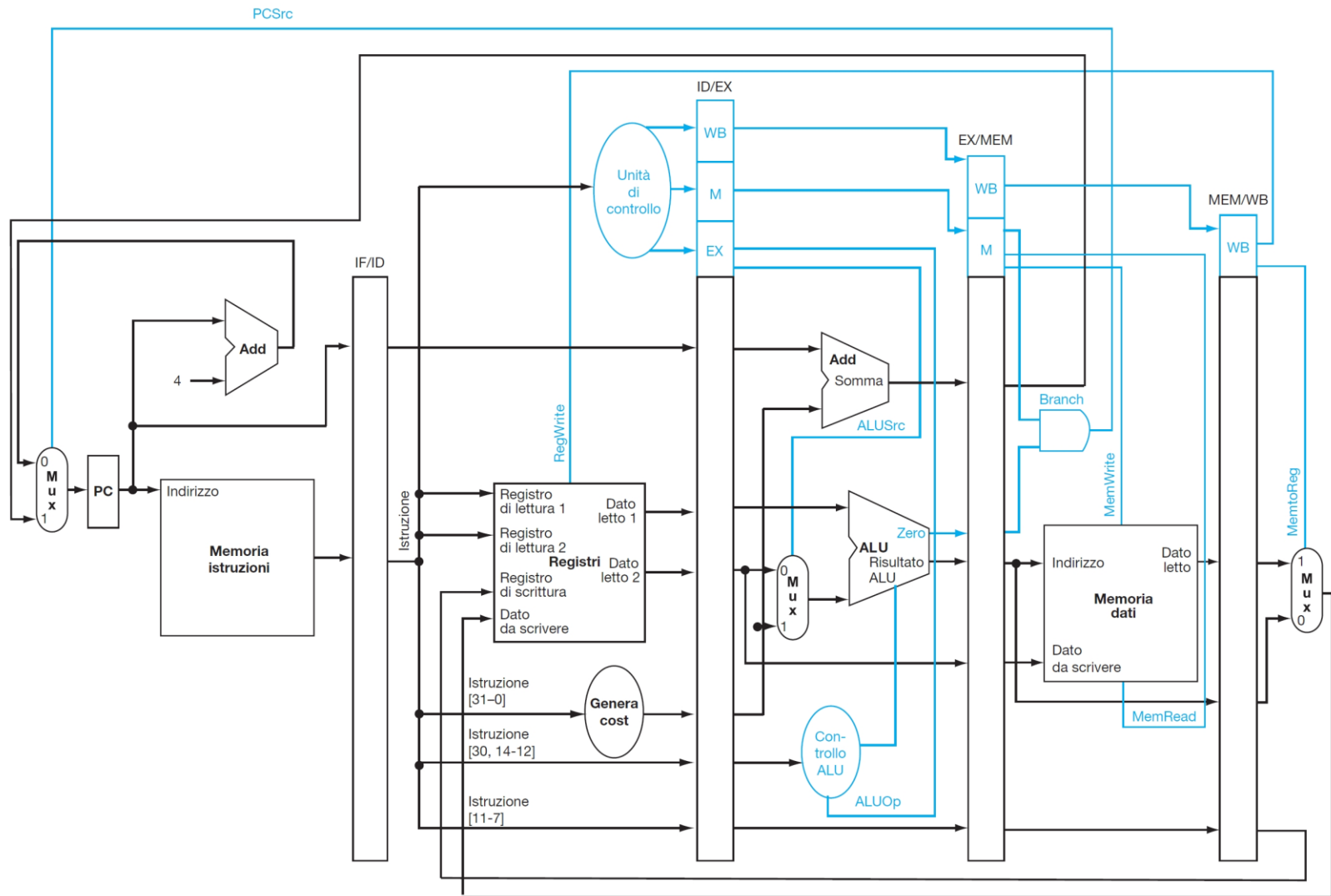
Anticipare le decisioni di branch



SAPIENZA
UNIVERSITÀ DI ROMA



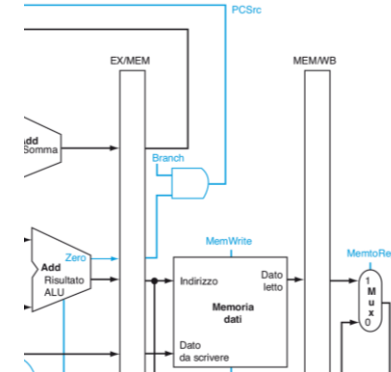
Decisione di branch presa in fase MEM (non ottimizzata)



Control hazards (beq)

L'istruzione **beq** potrebbe usare la ALU per fare il confronto, per cui:

- La decisione di salto potrebbe avvenire **dopo** la fase EXE (nella fase MEM)
 - in caso di salto, le **3 istruzioni seguenti** (già caricate) vanno annullate
- necessita degli argomenti nella fase EXE
 - quando si usa il forwarding, può aver bisogno comunque di uno **stallo** prima dell'istruzione di branch solo se **preceduta da 1w** (come ogni istruzione di tipo R)



lw t0, offset(t1) IF | ID | EX | MM | WB

add t0, t1, t2 -> IF | ID | EX | MM | WB

Come **annullare le istruzioni** a monte (a sx) in pipeline (istruzioni future)

- IF/ID. Istruzione viene azzerata → **NOP (No Operation)**
- ID/EXE. MemWrite e ID/EXE. RegWrite vengono azzerate
 - per non modificare memoria o registri (in realtà anche MemRead)

Pseudo-istruzione che non fa niente, come implementarla?

Text Segment				
Bkpt	Address	Code	Basic	Source
☐	0x00400000	0x00000013 addi x0,x0,0		1: nop
☐	0x00400004	0x00730263 beq x6,x7,4		2: beq t1,t2,label
☐				3: label:
☐	0x00400008	0x00000013 addi x0,x0,0		4: nop

istruzione di tipo R, legge 0 e "scrive" su x0

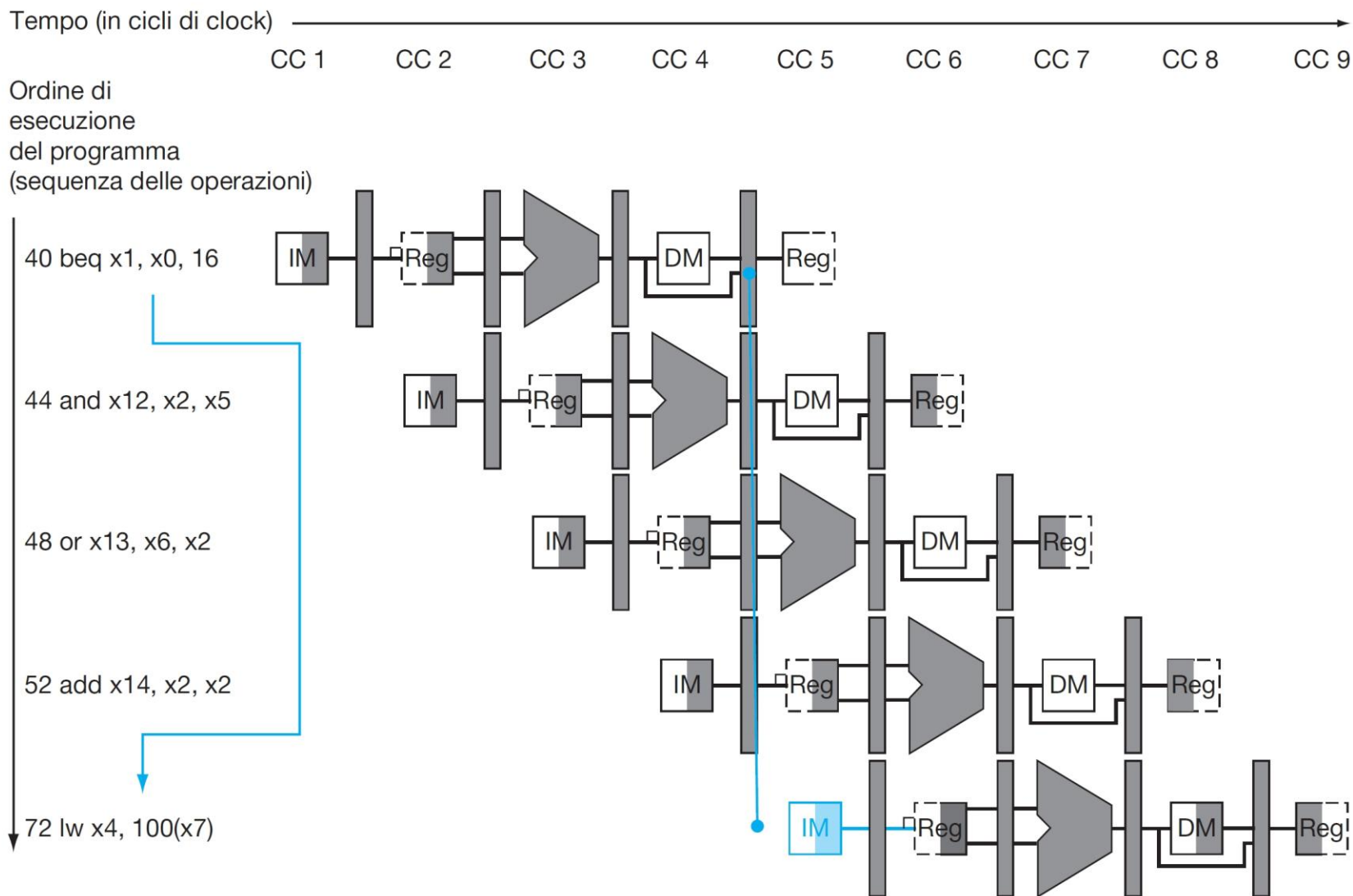
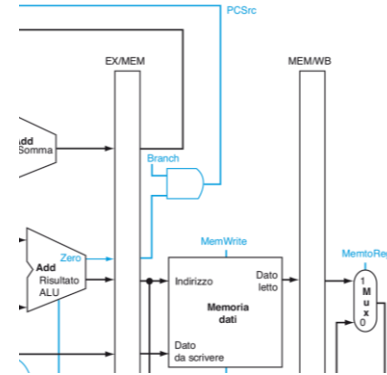


Figura 4.63 Impatto della pipeline sulle istruzioni di salto condizionato. I numeri alla sinistra di ciascuna istruzione (40, 44, ...) rappresentano l'indirizzo delle istruzioni. Dato che l'istruzione di salto condizionato decide se occorre saltare nello stadio MEM, corrispondente al quarto ciclo di clock dell'istruzione beq, le tre istruzioni che la seguono vengono comunque prelevate dalla memoria istruzioni e viene avviata la loro esecuzione. Se non si intervenisse, l'esecuzione di queste tre istruzioni inizierebbe prima che beq salti eventualmente alla lw contenuta all'indirizzo 72. Nella Figura 4.29 si supponeva che esistesse dell'hardware aggiuntivo per ridurre l'hazard sul controllo a un solo ciclo di clock, mentre in questa figura viene rappresentata l'unità di elaborazione non ottimizzata.

Control hazards (beq)

L'istruzione **beq** potrebbe **usare la ALU** per fare il confronto, per cui:

- La decisione di salto potrebbe avvenire **dopo** la fase **EXE** (nella fase **MEM**)
 - in caso di salto, le **3 istruzioni seguenti** (già caricate) vanno annullate
- necessita degli argomenti nella fase **EXE**
 - quando si usa il forwarding, può aver bisogno comunque di uno **stallo** prima dell'istruzione di branch solo se **preceduta da Tw** (come ogni istruzione di tipo R)



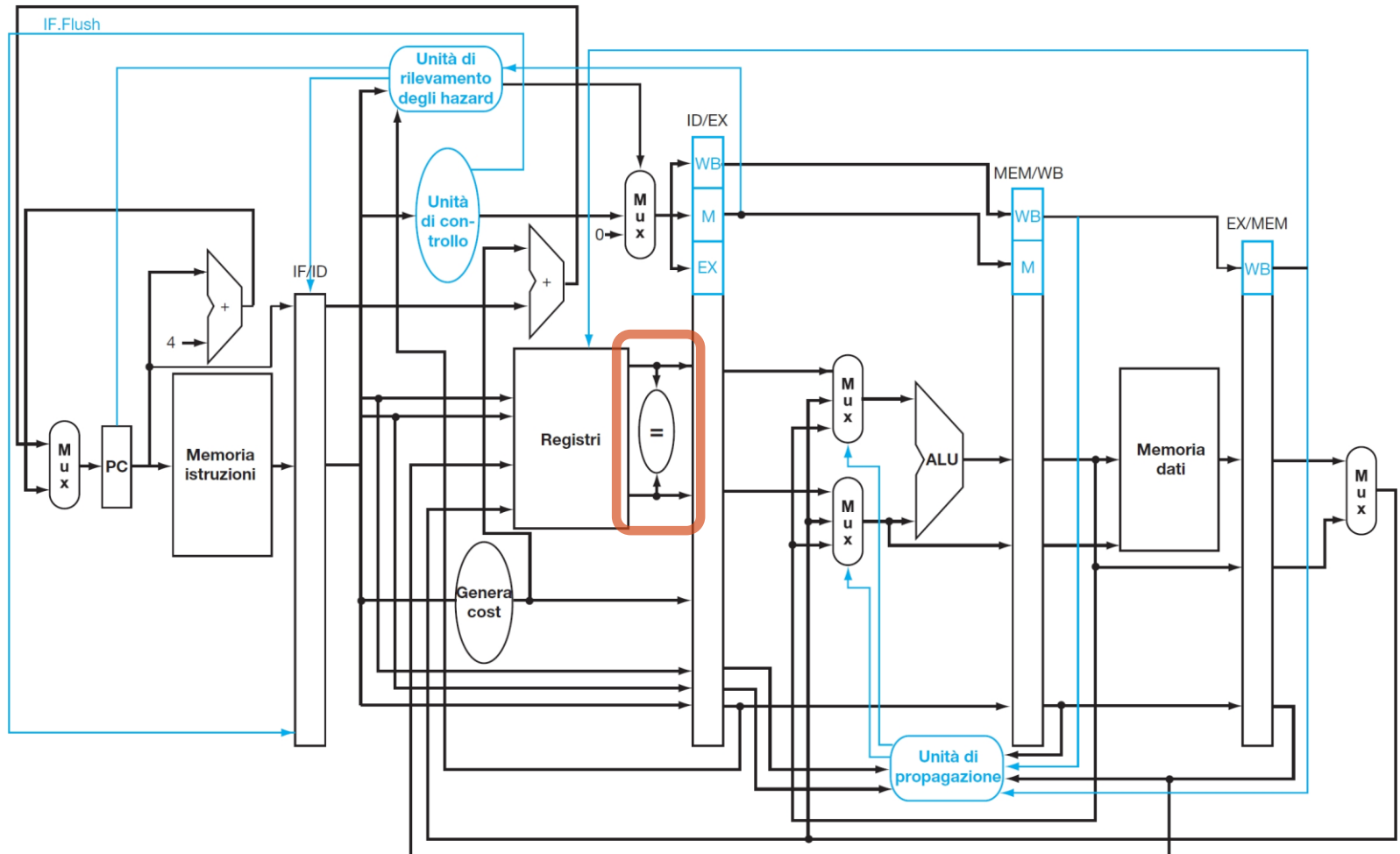
Come **annullare le istruzioni** a monte (a sx) in pipeline (istruzioni future)

- IF/ID. Istruzione viene azzerata → **NOP (No Operation)**
- ID/EXE. MemWrite e ID/EXE. RegWrite vengono azzerate
 - per non modificare memoria o registri (in realtà anche MemRead)

Per **anticipare la decisione di salto** alla fase **ID** occorre non usare la ALU:

- inserendo un comparatore tra i due argomenti letti dal blocco registri
- spostando la **logica di salto** ed il **calcolo del salto relativo** dalla fase EXE alla **fase ID**
- sarà necessaria una **unità di forwarding apposita** per la fase ID

Anticipiamo beq (e bne) alla fase ID

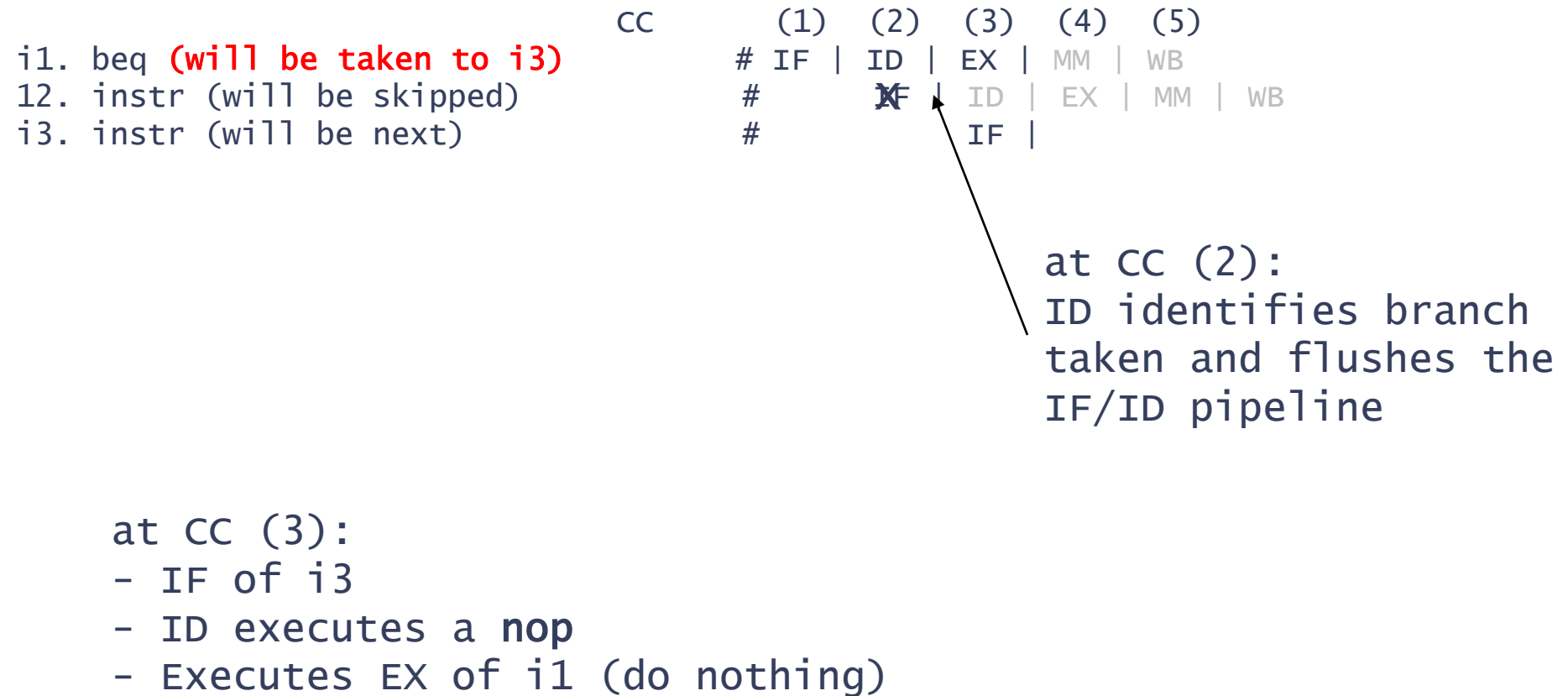


IF.Flush per branch in ID

	CC	(1)	(2)	(3)	(4)	(5)
i1. beq (will be taken to i3)	#	IF	ID	EX	MM	WB
i2. instr (will be skipped)	#		IF	ID	EX	MM
i3. instr (will be next)						WB

at CC (2):
ID identifies branch
taken and flushes the
IF/ID pipeline

IF.Flush per branch in ID



Anticipando la decisione del branch dalla fase MEM alla fase ID si passa da 3 stalli a uno (non eliminabile) quando il branch è taken

Anticipiamo beq (e bne) alla fase ID

L'abbassamento del numero di stalli nel caso di predizione sbagliata (da 3 a 1) non è gratuito.

La fase in cui beq e bne necessitano dei dati viene anticipata da EXE a ID.

Nel caso di lw con data hazard quindi ci saranno due stalli non risolvibili con ulteriori forwarding:



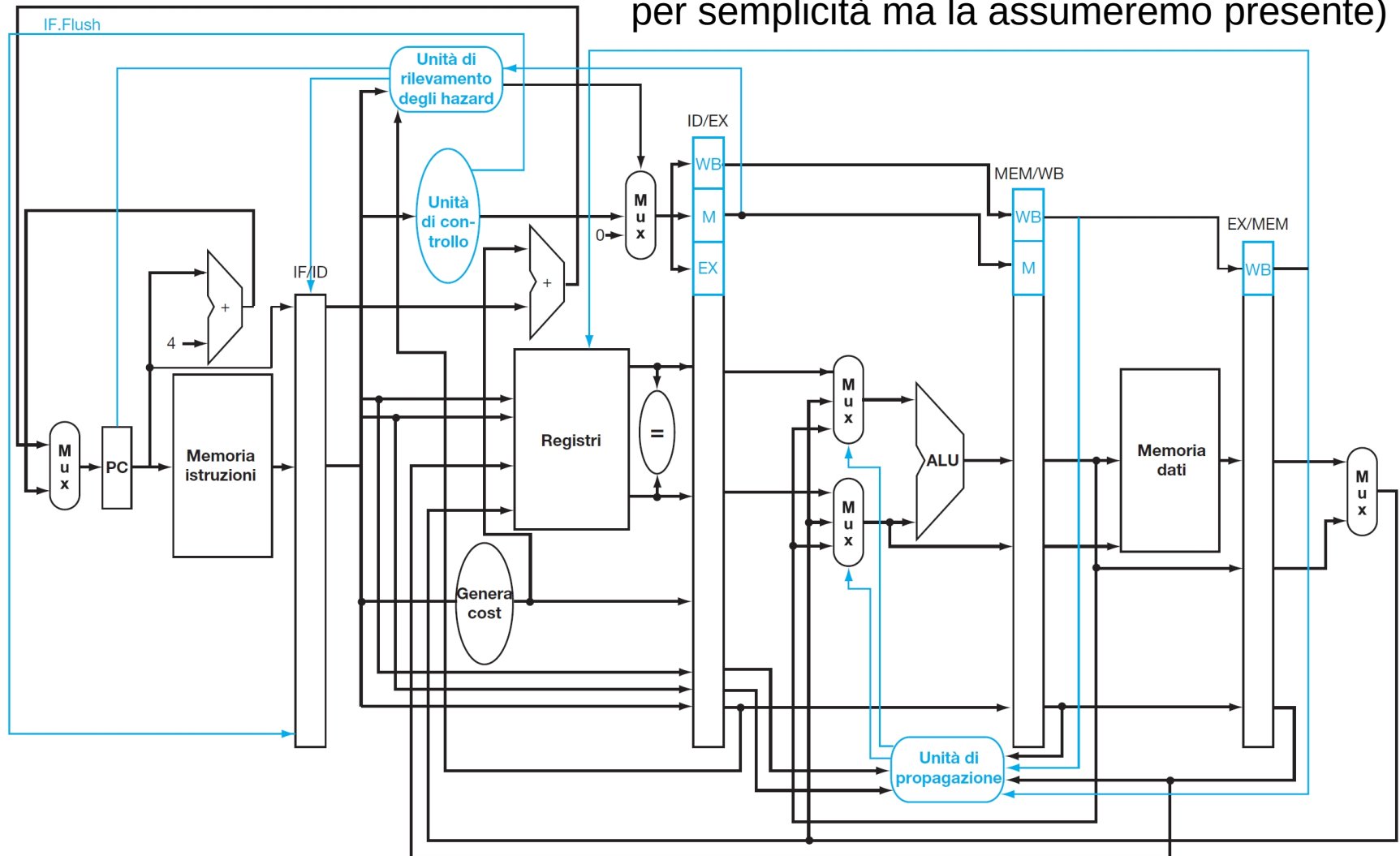
Anche operazioni di tipo R seguite da beq o bne richiedono uno stallo per queste ultime (in caso di data hazard) usando un'unità di forwarding EX -> ID (altrimenti due stalli):



Questi stalli (come per lw->R) vengono gestiti dall'unità rilevamento Data Hazard in fase di ID (ripete l'istruz al ciclo di clock successivo)

Architettura della CPU con pipeline

Inoltre per il forwarding EXE -> ID serve un'altra unità di propagazione (non rappresentata qui per semplicità ma la assumeremo presente)



Predire i salti



SAPIENZA
UNIVERSITÀ DI ROMA

salto ritardato (recap)

idea:

inserire un'istruzione (o due) che verrà eseguita in entrambi i casi (salto o no) per evitare di dover inserire stalli dopo un branch

se l'istruzione che segue la beq venisse **SEMPRE** eseguita anche se il salto fosse effettuato, si elimina di fatto lo stallo eseguendo quell'istruzione al posto dello stallo (non sempre possibile)

Impatto della beq (con tutte unità di forwarding)

Se il salto venisse deciso in fase **MEM** (adoperando la ALU) si hanno

- **3 stalli dopo**, se il salto venisse eseguito (control hazard, eliminabili con **tre salti ritardati**)
- **1 stallo prima** se la **beq** fosse preceduta da **lw** (data hazard)
 - riordinando il codice, si può ovviare a questo stallo (non sempre)

Se la **beq** fosse spostata nella **fase ID**:

- **1 stallo dopo**, se il salto fosse eseguito (eliminabile con un **salto ritardato**)
- **2 stalli prima** (in caso di data hazard) se preceduta da **lw** o **1 stallo** se preceduta da istruzioni di tipo **R**
 - riordinando il codice, si può ovviare a questi stalli (non sempre)

L'impatto dello stallo dopo l'istruzione di branch dipende dal tipo di codice che usa la **beq**, soprattutto nei cicli:

- se il test è all'inizio, seguito dal corpo del ciclo, la **beq** fa **un solo salto alla fine del ciclo**
- se il test è alla fine, dopo il corpo, la **beq** effettua **molti salti** e in un solo caso continuerà

La CPU vista finora presume che il salto non venga effettuato (**branch not taken by default**)

- se si potesse «prevedere» il salto si potrebbero caricare le istruzioni giuste ed evitare stalli

Default (esame): decisione in **ID** senza salto ritardato, tutte unità di forwarding attive

Riepilogo forwarding con tutte le unità che useremo

- Forwarding da EXE -> EXE

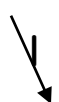
- Serve per istruzioni di tipo R, lw o sw quando c'è un data hazard

add t0, t1, t2	IF	ID	EX	MM	WB
add t4, t0, t1	IF	ID	EX	MM	WB

- Forwarding da MEM -> EXE

- Serve per istruzioni di tipo R, lw o sw quando c'è un data hazard (più distante)

add t0, t1, t2	IF	ID	EX	MM	WB
sub t3, t1, t2	IF	ID	EX	MM	WB
add t4, t0, t1	IF	ID	EX	MM	WB



- Forwarding da EXE -> ID

- Serve per istruzioni di tipo branch quando c'è un data hazard

add t0, t1, t2	IF	ID	EX	MM	WB	
beq t0, x0, label	->	IF	ID	EX	MM	WB

(il forwarding da MEM a ID non avrebbe senso perché possiamo leggere direttamente da WB)

Impatto della beq (senza unità di forwarding)

Se il salto viene deciso in fase **MEM** (adoperando la ALU) si hanno

- **3 stalli dopo**, se il salto viene eseguito (control hazard, eliminabili con **tre salti ritardati**)
- **2 stalli prima** se la **beq** è preceduta da **lw** (data hazard)
 - riordinando il codice, si può ovviare a questi stallo (non sempre)

Se la **beq** è spostata nella **fase ID**:

- **1 stallo dopo**, se il salto viene eseguito (eliminabile con un **salto ritardato**)
- **2 stalli prima** (in caso di data hazard)
 - riordinando il codice, si può ovviare a questi stalli (non sempre)

L'impatto dello stallo dopo l'istruzione di branch dipende dal tipo di codice che usa la **beq**, soprattutto nei cicli:

- se il test è all'inizio, seguito dal corpo del ciclo, la **beq** fa **un solo salto alla fine del ciclo**
- se il test è alla fine, dopo il corpo, la **beq** effettua **molti salti** e in un solo caso continuerà

La CPU vista finora presume che il salto non venga effettuato (**branch not taken by default**)

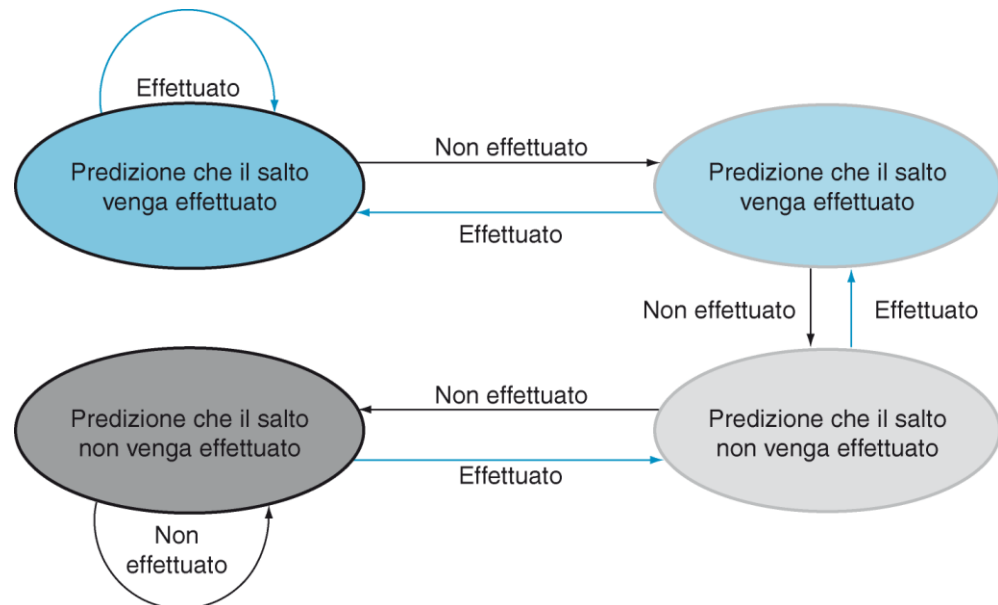
- se si potesse «prevedere» il salto si potrebbero caricare le istruzioni giuste ed evitare stalli

Default (esame): decisione in ID senza salto ritardato

Predizione dei salti (livello HW): default branch (not?) taken

Ad ogni istruzione di salto possiamo associare **bit che «contano»** i salti già effettuati per decidere se sia più probabile che il salto venga effettuato o meno

- indice in micro-memoria indicizzata dagli LSB dell'istruzione
 - problema di «hash collision» (istruzioni con stessi LSB ma diversi comportamenti)
- con **un bit** si può rappresentare l'informazione «l'ultima volta il salto è stato effettuato»
 - nel realizzare un ciclo la previsione sarà sbagliata 2 volte: **entrando e uscendo**
- con **due bit** si può realizzare una macchina a stati finiti
 - necessita di due errori consecutivi per cambiare previsione
 - quindi **incorre in meno errori** nei cicli a forte prevalenza di uno specifico tipo di scelta (una sola previsione errata)



Ritardare il salto

Oppure per recuperare il tempo perso dallo stallo si può **ritardare il salto**

- eseguire **in ogni caso** l'istruzione che segue la beq (**salto ritardato o delay slot**)

Questa strategia richiede

- una scrittura del codice assembly diversa, oppure
- l'uso di un compilatore capace di automatizzare le necessarie modifiche al codice macchina prodotto

Nel delay slot è possibile copiare una delle istruzioni che vanno sempre eseguite:

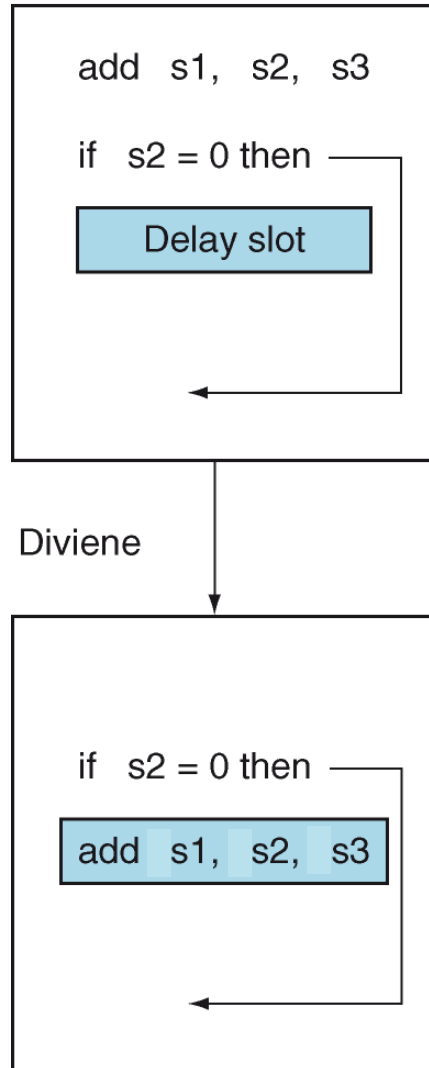
- **Caso 1: un'istruzione precedente** che non abbia dipendenze (anche indirette) con la beq

Se non ci sono istruzioni precedenti senza dipendenze:

- **Caso 2: l'istruzione alla destinazione del salto (non sempre possibile)**
 - NOTA: l'istruzione viene **copiata** perché potrebbe far parte di altri flussi di controllo
 - NOTA: se il salto NON viene effettuato, l'istruzione scelta (sempre eseguita) non deve creare problemi:
 - ad esempio, può calcolare un valore non più necessario nel codice seguente
 - l'importante è che non sia dannosa per l'esecuzione successiva
- **Caso 3: inserisco una nop se non ci sono alternative**

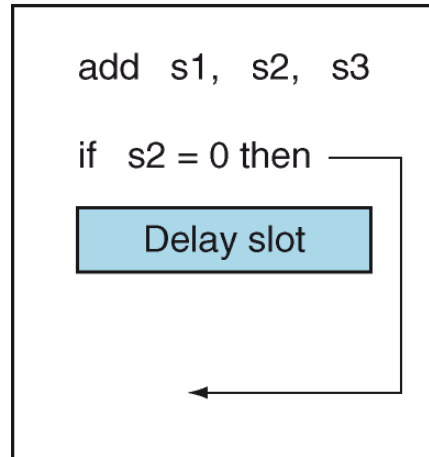
Riordinamento con salto ritardato (livello assemblatore)

a. Da prima

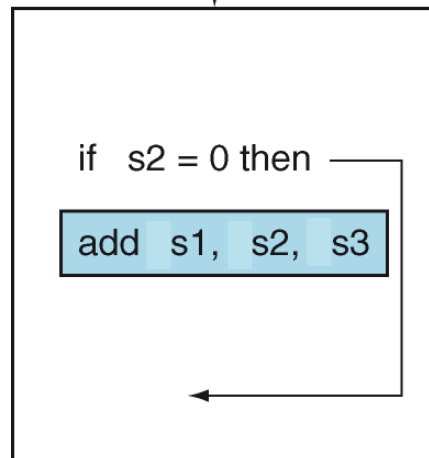


Riordinamento con salto ritardato (livello assembler)

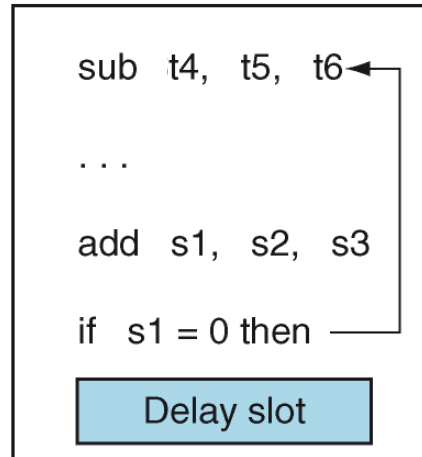
a. Da prima



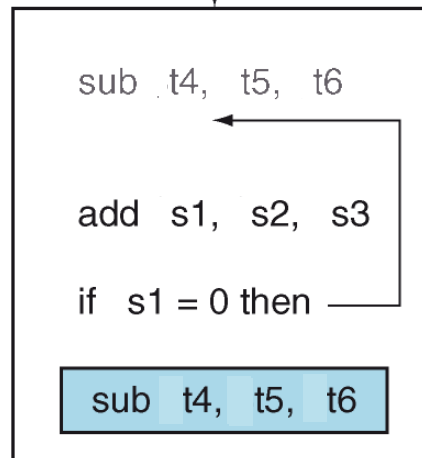
Diviene



b. Dall'indirizzo di salto

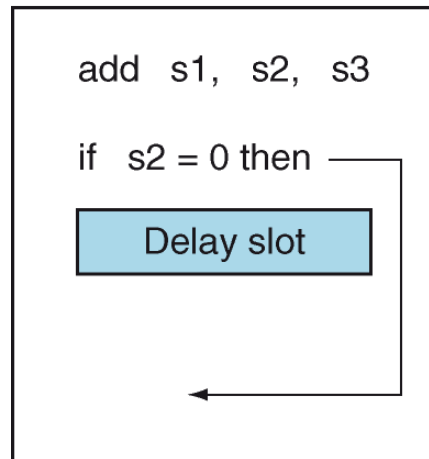


Diviene

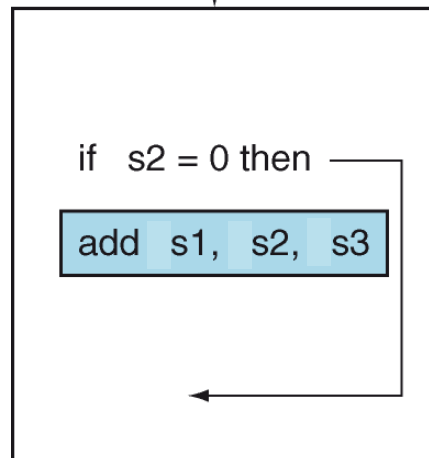


Riordinamento con salto ritardato (livello assemblatore)

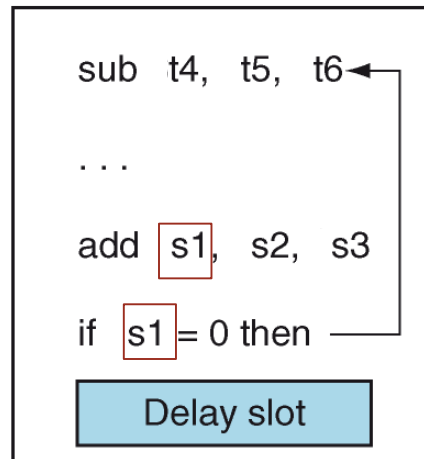
a. Da prima



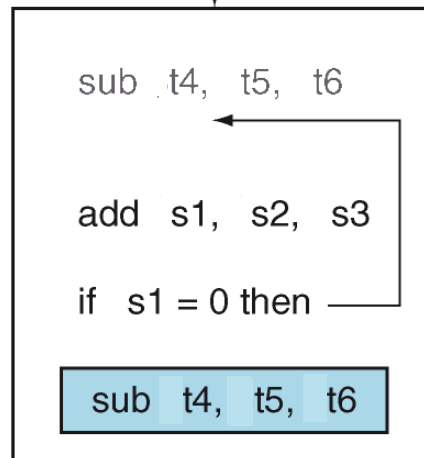
Diviene



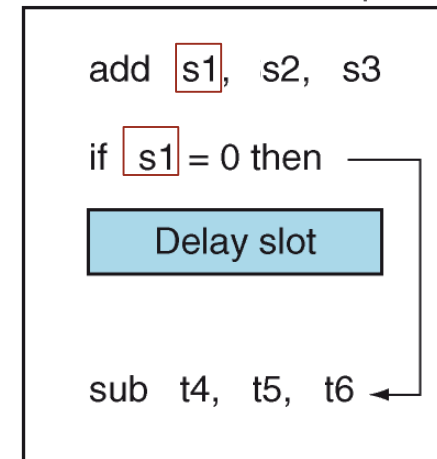
b. Dall'indirizzo di salto



Diviene



c. Dall'indirizzo di salto, nel caso di fallimento della predizione



Diviene

